



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

OPERATING SYSTEMS AND SYSTEMS PROGRAMMING (25CS2104E)

2026–27, Term-I

Project Problem Statement Submission Form

Section / Team Details

Section No.: 1

Team No.: 7

Project Title: Linux Process Creation, Execution and Termination System

Team Members

Roll Number	Student Name	Signature
2520030607	Ili.Harshika	
2520030512	Gurrapu Nikshitha	
2520030365	Angela Jose	

1. Abstract

Process management is an important function of an Operating System, as it involves creating, executing, synchronizing, and terminating processes. This project proposes a Linux Process Creation, Execution and Termination System developed using the C programming language and Linux/POSIX system calls. The system demonstrates the basic life cycle of a process in a practical way.

The system creates a child process from a parent process using the `fork()` system call. The child process executes a selected program using the `exec()` family of system calls, while the parent process waits for the child to complete using `wait()`. After the child process terminates, the parent retrieves and displays the termination status. The project provides a clear practical understanding of parent-child relationships, process creation, program execution, process synchronization, and process termination in a Linux environment.

2. Problem Statement

This project aims to develop a Linux-based system that practically demonstrates the complete process life cycle. The system will create a child process from a parent process using `fork()`, execute a selected program in the child using the `exec()` family of system calls, and allow the parent process to wait for the child using `wait()`. After the child completes its execution and terminates, the system will retrieve and display its termination status. This provides a practical understanding of process creation, execution, synchronization, and termination in Linux.

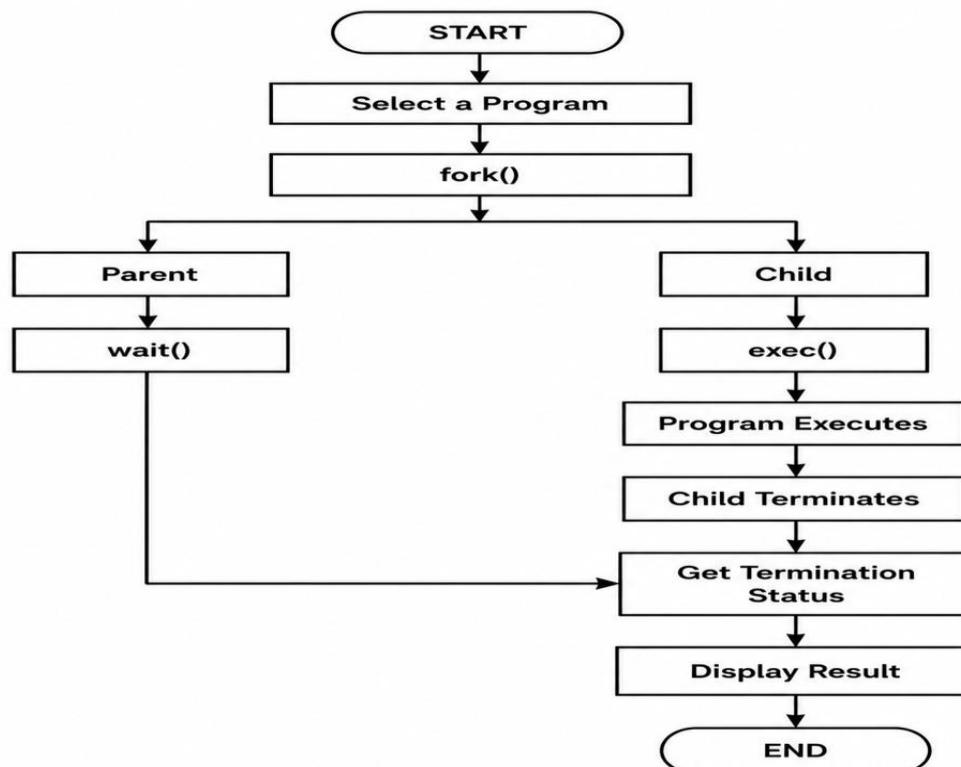
3. Objectives

- i. To implement process creation in Linux using the `fork()` system call and understand the parent-child process relationship.
- ii. To execute a selected program in the child process using the `exec()` family of system calls.
- iii. To synchronize the parent and child processes using the `wait()` system call.
- iv. To demonstrate process termination and retrieve and display the termination status of the child process.

4. Proposed Methodology

The system is implemented in C on a Linux platform using standard POSIX system calls. Execution begins when the user selects a program to run. The main process then invokes `fork()` to create a child process, splitting execution into two paths. The parent process calls `wait()` and blocks until the child finishes, while the child process calls an `exec()` function to replace its process image with the selected program and run it to completion. Once the program finishes, the child process terminates, and the parent, unblocked from `wait()`, retrieves the child's exit/termination status. Finally, the result is displayed to the user, showing whether the program executed successfully or failed. The overall project flow is illustrated below.

➤ Project Overflow :



5. Operating Systems Concepts / Linux APIs Used

OS Concept / Linux API / System Call	Purpose in the Project
fork()	Creates a new child process (duplicate of the parent process)
exec() family (execvp/execl)	Replaces the child process image with the selected program
wait() / waitpid()	Suspends the parent until the child process terminates; synchronizes parent and child
Process ID (PID, PPID)	Uniquely identifies parent and child processes
Exit status / WIFEXITED, WEXITSTATUS	Used to retrieve and interpret the child's termination status
Process Control Block (PCB) concept	Underlying OS structure that maintains process state, used to explain process lifecycle

6. Individual Contribution

Roll Number	Student Name	Individual Responsibility
2520030607	Ili.Harshika	Implement process creation using fork() and handle the parent-child process relationship.
2520030512	Gurrapu Nikshitha	Implement program execution using exec() and process synchronization using wait().
2520030365	Angela Jose	Implement process termination and exit-status handling, and perform testing of the system.

7. Tools / Platforms / Software Used

Tool / Platform / Software	Purpose
Linux / Ubuntu	Base operating system on which the program is developed and executed
C / C++	Programming language used to implement fork(), exec(), and wait() calls
GCC Compiler	Used to compile the C source code into an executable
GDB / Terminal	Used for debugging and running/testing the program
VS Code	Code editor used for writing and organizing the project source code

8. Expected Outcome

The system is expected to successfully create a child process using fork(), execute a user-selected program in the child using the exec() family of calls, and have the parent process synchronize with the child using wait(). Upon completion, the system should correctly retrieve and display the child process's termination status, giving a clear, working demonstration of process creation, execution, synchronization, and termination on a Linux system.

9. GitHub Repository

GitHub Repository Link: https://github.com/iliharshika/SystemForge_OSSP

https://github.com/gurrapunikshitha/SystemForge_OSSP

https://github.com/angelapvjose92/SystemForge_OSSP

Repository Created: Yes

Faculty Name and Signature

Faculty Name: Dr.D.Bheekya

Signature: _____

Remarks: _____

Faculty Use

Project Approved: _____

☐ Yes ☐ No

Date: _____

Faculty Signature: _____